

Department of Mathematics · University of Padua

Architectural Trade-offs in Zero-Knowledge Proof Decomposition

A Traveling-Salesman Case Study

CANDIDATE

Leonardo Callegari

Master's Degree in Cybersecurity



THESIS DEFENSE

September 23, 2026

SUPERVISOR

Prof. Mauro Conti

University of Padua

EXTERNAL SUPERVISOR

Prof. Martin Ochoa

zkSecurity · Applied Cryptography

A logistics scenario with a privacy problem

COMPANY



PROVIDER



Requires

- Coverage
- $\text{Cost} \leq \text{Threshold}$

Holds

- Private route
- Exact cost

PUBLIC LOCATIONS



A logistics scenario with a privacy problem

COMPANY



PROVIDER



Requires

- Coverage
- $\text{Cost} \leq \text{Threshold}$

Holds

- Private route
- Exact cost

PUBLIC LOCATIONS



Can we prove compliance without disclosure?

Zero-Knowledge Proofs

Formalizing the problem

PUBLIC INSTANCE

Complete directed graph $G = (\mathcal{N}, \mathcal{A})$ with $n = |\mathcal{N}|$ nodes.

- $\mathcal{N} = \{0, \dots, n-1\}$, $\mathcal{A} = \{(i, j) : i, j \in \mathcal{N}, i \neq j\}$
- $n \times n$ cost matrix $C = [c(i, j)]_{i, j \in \mathcal{N}}$, cost threshold T

MAP $\rightarrow$ GRAPH • ROUTE $\rightarrow$ HAMILTONIAN CYCLE



PROVIDER $\rightarrow$ PROVER • COMPANY $\rightarrow$ VERIFIER

Arcs are omitted. Only the selected Hamiltonian cycle π is drawn.

Formalizing the problem



DIPARTIMENTO
MATEMATICA



UNIVERSITÀ
DI PADOVA

PUBLIC INSTANCE

Complete directed graph $G = (\mathcal{N}, \mathcal{A})$ with $n = |\mathcal{N}|$ nodes.

- $\mathcal{N} = \{0, \dots, n-1\}$, $\mathcal{A} = \{(i, j) : i, j \in \mathcal{N}, i \neq j\}$
- $n \times n$ cost matrix $C = [c(i, j)]_{i, j \in \mathcal{N}}$, cost threshold T

PRIVATE ROUTE (WITNESS)

Hamiltonian cycle

- Ordered sequence $\pi = (\pi_0, \pi_1, \dots, \pi_{n-1})$
- Implicit closing arc $\pi_n = \pi_0$
- $\text{cost}(\pi) = \sum_{i=0}^{n-1} c(\pi_i, \pi_{i+1})$

MAP $\rightarrow$ GRAPH • ROUTE $\rightarrow$ HAMILTONIAN CYCLE



PROVIDER $\rightarrow$ PROVER • COMPANY $\rightarrow$ VERIFIER

Arcs are omitted. Only the selected Hamiltonian cycle π is drawn.

Formalizing the problem



PUBLIC INSTANCE

Complete directed graph $G = (\mathcal{N}, \mathcal{A})$ with $n = |\mathcal{N}|$ nodes.

- $\mathcal{N} = \{0, \dots, n-1\}$, $\mathcal{A} = \{(i, j) : i, j \in \mathcal{N}, i \neq j\}$
- $n \times n$ cost matrix $C = [c(i, j)]_{i, j \in \mathcal{N}}$, cost threshold T

PRIVATE ROUTE (WITNESS)

Hamiltonian cycle

- Ordered sequence $\pi = (\pi_0, \pi_1, \dots, \pi_{n-1})$
- Implicit closing arc $\pi_n = \pi_0$
- $\text{cost}(\pi) = \sum_{i=0}^{n-1} c(\pi_i, \pi_{i+1})$

CLAIM

$$R = \{ \langle (C, T), \pi \rangle : \pi \text{ is a permutation of } \mathcal{N} \text{ and } \text{cost}(\pi) \leq T \}$$

MAP $\rightarrow$ GRAPH • ROUTE $\rightarrow$ HAMILTONIAN CYCLE



PROVIDER $\rightarrow$ PROVER • COMPANY $\rightarrow$ VERIFIER

Arcs are omitted. Only the selected Hamiltonian cycle π is drawn.

Assumption: prover already holds a route.

A zk-SNARK proves the claim without revealing the witness



A zk-SNARK provides

- **Short proof:** fast verification.
- **Single message:** no online interaction.
- **Valid witness knowledge:** not just existence.
- **Generality:** many relations, same machinery.

Relation $\rightarrow$ Circuit $\rightarrow$ Proof

Public inputs + private witness

Succinct verification does not mean cheap proving

- Prover resources scale with instance size n : circuit size, peak memory, proving time.
- On fixed hardware, proof generation for a sufficiently large circuit eventually fails.

A zk-SNARK proves the claim without revealing the witness



A zk-SNARK provides

- **Short proof:** fast verification.
- **Single message:** no online interaction.
- **Valid witness knowledge:** not just existence.
- **Generality:** many relations, same machinery.

Relation $\rightarrow$ Circuit $\rightarrow$ Proof

Public inputs + private witness

Succinct verification does not mean cheap proving

- Prover resources scale with instance size n : circuit size, peak memory, proving time.
- On fixed hardware, proof generation for a sufficiently large circuit eventually fails.

Can proof decomposition improve scalability?

A zk-SNARK proves the claim without revealing the witness



A zk-SNARK provides

- **Short proof:** fast verification.
- **Single message:** no online interaction.
- **Valid witness knowledge:** not just existence.
- **Generality:** many relations, same machinery.

Relation $\rightarrow$ Circuit $\rightarrow$ Proof

Public inputs + private witness

Succinct verification does not mean cheap proving

- Prover resources scale with instance size n : circuit size, peak memory, proving time.
- On fixed hardware, proof generation for a sufficiently large circuit eventually fails.

RESEARCH QUESTION

How do monolithic and composite zk-proof architectures compare for globally constrained statements, and what trade-offs arise from decomposition and recombination?

Encoding the monolithic relation

RELATION

$$R = \{ \langle (C, T), \pi \rangle : \pi \text{ is a permutation of } \mathcal{N} \text{ and } \text{cost}(\pi) \leq T \}$$

CONSTRAINTS**DIRECT CONSTRUCTION****REFINED CONSTRUCTION*****COVERAGE******COST BINDING******THRESHOLD******PROOF INTERFACE***

Encoding the monolithic relation

RELATION

$$R = \{ \langle (C, T), \pi \rangle : \pi \text{ is a permutation of } \mathcal{N} \text{ and } \text{cost}(\pi) \leq T \}$$

CONSTRAINTS**DIRECT CONSTRUCTION****REFINED CONSTRUCTION****COVERAGE**

- Range checks
- Pairwise distinctness

COST BINDING

Direct lookup into full cost matrix C

$$\bullet \ c(\pi_i, \pi_{i+1}) = C[\pi_i \cdot n + \pi_{i+1}]$$

THRESHOLD

$$\text{cost}(\pi) = \sum_{i=0}^{n-1} c(\pi_i, \pi_{i+1}) \leq T$$

PROOF INTERFACE

Full cost matrix as public input

$(C, T) \longrightarrow n^2 + 1$ public values

Encoding the monolithic relation

RELATION

$$R = \{ \langle (C, T), \pi \rangle : \pi \text{ is a permutation of } \mathcal{N} \text{ and } \text{cost}(\pi) \leq T \}$$

CONSTRAINTS**DIRECT CONSTRUCTION****REFINED CONSTRUCTION****COVERAGE**

- Range checks
- Pairwise distinctness

Permutation check through sorting

$$\bullet \text{ sort}(\pi) \stackrel{?}{=} \{0, \dots, n-1\} = \mathcal{N}$$

COST BINDING

Direct lookup into full cost matrix C

$$\bullet c(\pi_i, \pi_{i+1}) = C[\pi_i \cdot n + \pi_{i+1}]$$

Merkle tree commitment + openings

- One root binds every matrix entry
- Openings authenticate costs

THRESHOLD

$$\text{cost}(\pi) = \sum_{i=0}^{n-1} c(\pi_i, \pi_{i+1}) \leq T$$

$$\text{cost}(\pi) = \sum_{i=0}^{n-1} c(\pi_i, \pi_{i+1}) \leq T$$

PROOF INTERFACE

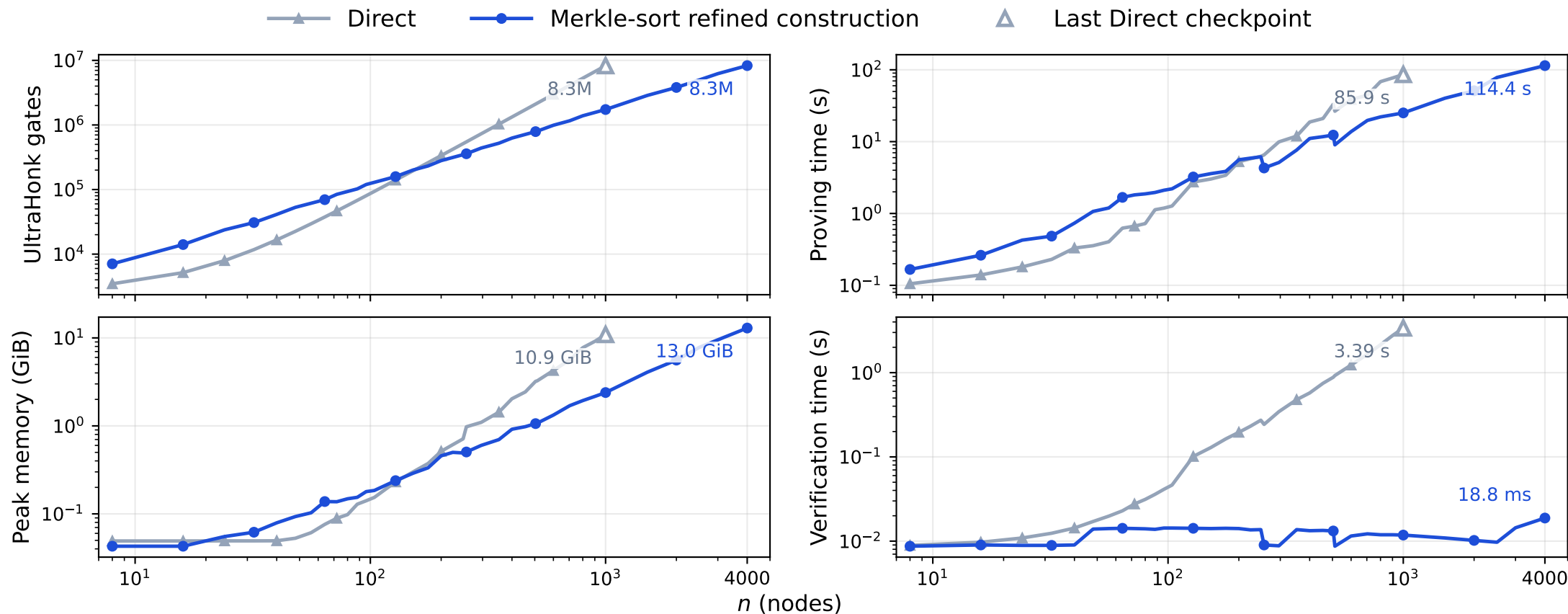
Full cost matrix as public input

$$(C, T) \longrightarrow n^2 + 1 \text{ public values}$$

Committed matrix Merkle root

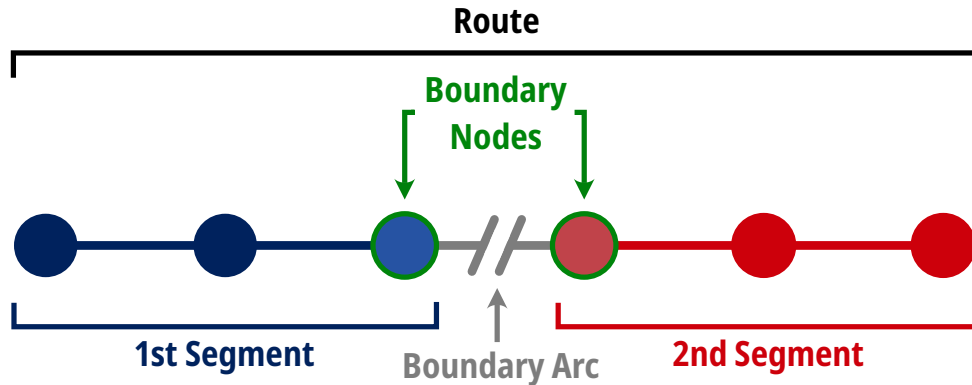
$$(\text{root}(C), T) \longrightarrow 2 \text{ public values}$$

One circuit eventually exhausts fixed hardware



All panels use logarithmic axes. Every point comes from an executed benchmark, with no extrapolation.

Segments shrink local tasks but lose the global view



Semantic cut at the route level

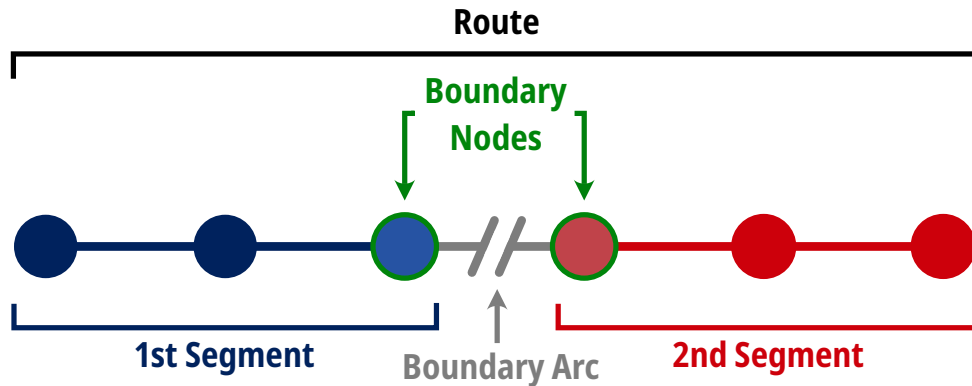
$$\pi = (\pi_0, \dots, \pi_{n-1}) = R_0 \parallel \dots \parallel R_{K-1}$$

$$|R_i| = M = n/K, \text{ assuming } K \mid n$$

Segment can establish

- Coverage evidence for its M nodes
- Cost binding for $M - 1$ internal arcs
- Partial cost L_i

Segments shrink local tasks but lose the global view



Semantic cut at the route level

$$\pi = (\pi_0, \dots, \pi_{n-1}) = R_0 \parallel \dots \parallel R_{K-1}$$

$$|R_i| = M = n/K, \text{ assuming } K \mid n$$

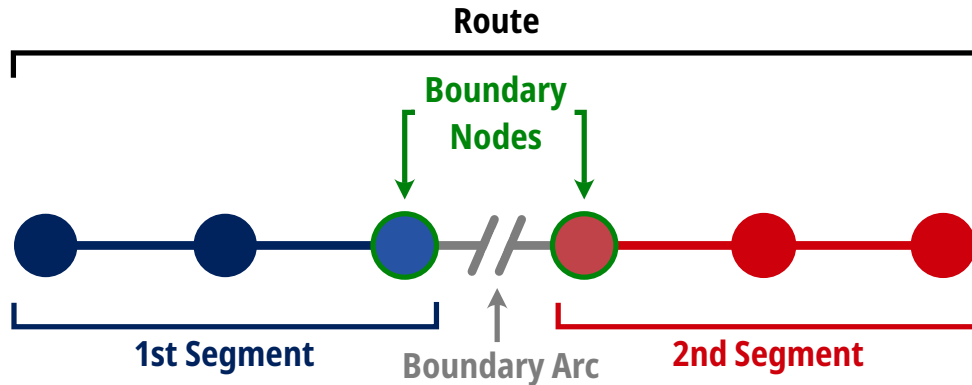
Segment can establish

- Coverage evidence for its M nodes
- Cost binding for $M - 1$ internal arcs
- Partial cost L_i

Segment **cannot** establish

- Complete coverage over n nodes
- Cost binding of K boundary arcs
- Complete threshold check
- **Consistency with other segments**

Segments shrink local tasks but lose the global view



Semantic cut at the route level

$$\pi = (\pi_0, \dots, \pi_{n-1}) = R_0 \parallel \dots \parallel R_{K-1}$$

$$|R_i| = M = n/K, \text{ assuming } K \mid n$$

Segment can establish

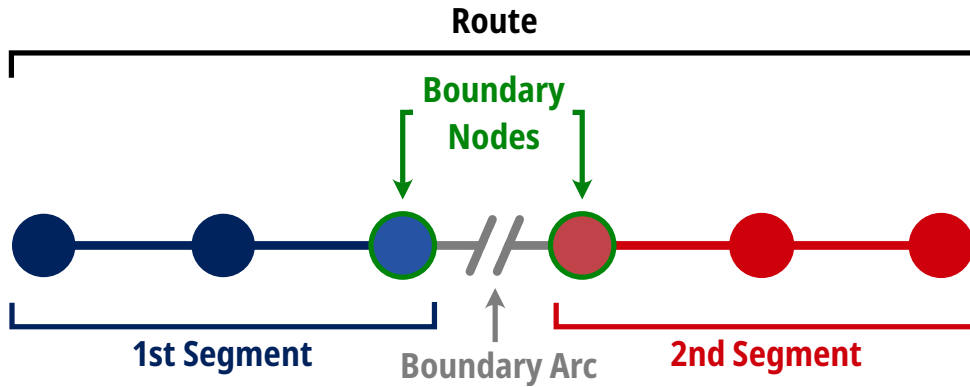
- Coverage evidence for its M nodes
- Cost binding for $M - 1$ internal arcs
- Partial cost L_i

Segment **cannot** establish

- Complete coverage over n nodes
- Cost binding of K boundary arcs
- Complete threshold check
- **Consistency with other segments**

Global constraints + composition binding for consistency $\Rightarrow$ Recombination

Recombination must restore the complete claim

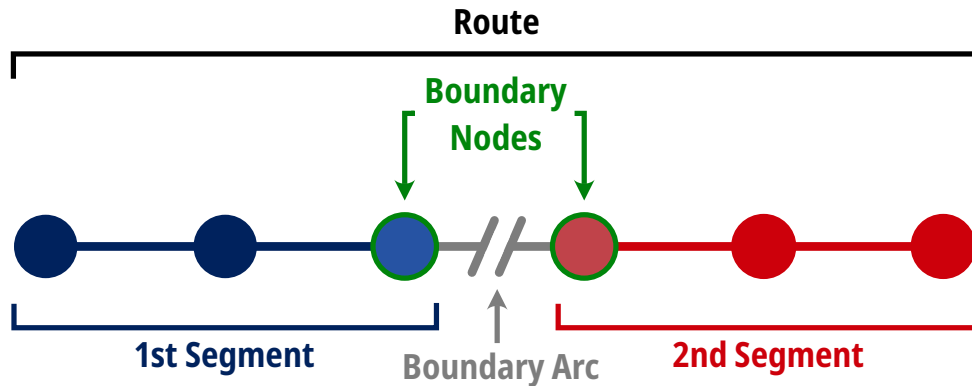


Certified segment summary

$$\sigma_i^{\text{seg}} = (\text{cov}_i, \text{start}_i, \text{end}_i, L_i)$$

$$b_i = C[\text{end}_i \cdot n + \text{start}_{i+1 \bmod K}]$$

Recombination must restore the complete claim



Certified segment summary

$$\sigma_i^{\text{seg}} = (\text{cov}_i, \text{start}_i, \text{end}_i, L_i)$$

$$b_i = C[\text{end}_i \cdot n + \text{start}_{i+1 \bmod K}]$$

Restores the global relation

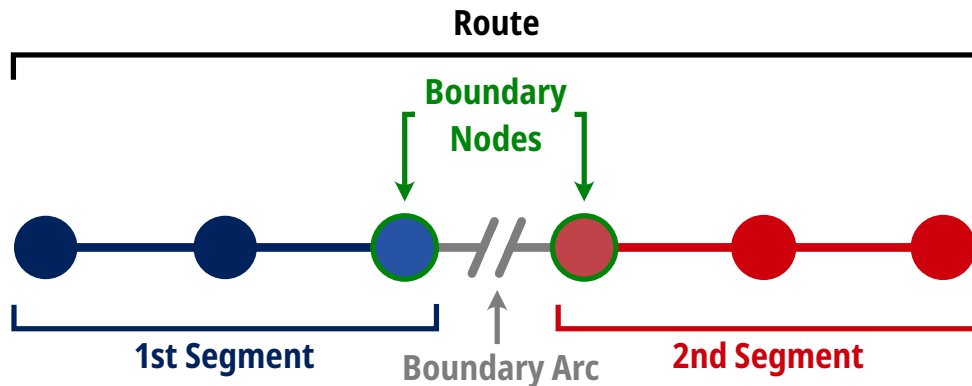
- Coverage over all n nodes through cov_i
- Binds K boundary arcs

$$\text{cost}(\pi) = \sum_{i=0}^{K-1} L_i + \sum_{i=0}^{K-1} b_i \leq T$$

Provides composition binding

- Common matrix root and intended threshold
- Each σ_i matches its segment proof

Recombination must restore the complete claim



Certified segment summary

$$\sigma_i^{\text{seg}} = (\text{cov}_i, \text{start}_i, \text{end}_i, L_i)$$

$$b_i = C[\text{end}_i \cdot n + \text{start}_{i+1 \bmod K}]$$

Restores the global relation

- Coverage over all n nodes through cov_i
- Binds K boundary arcs

$$\text{cost}(\pi) = \sum_{i=0}^{K-1} L_i + \sum_{i=0}^{K-1} b_i \leq T$$

Provides composition binding

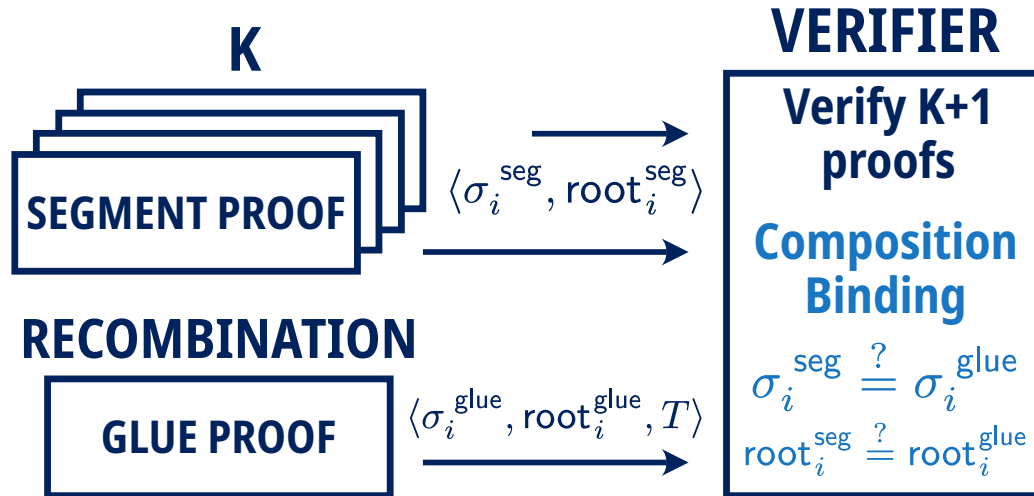
- Common matrix root and intended threshold
- Each σ_i matches its segment proof

Coverage Evidence

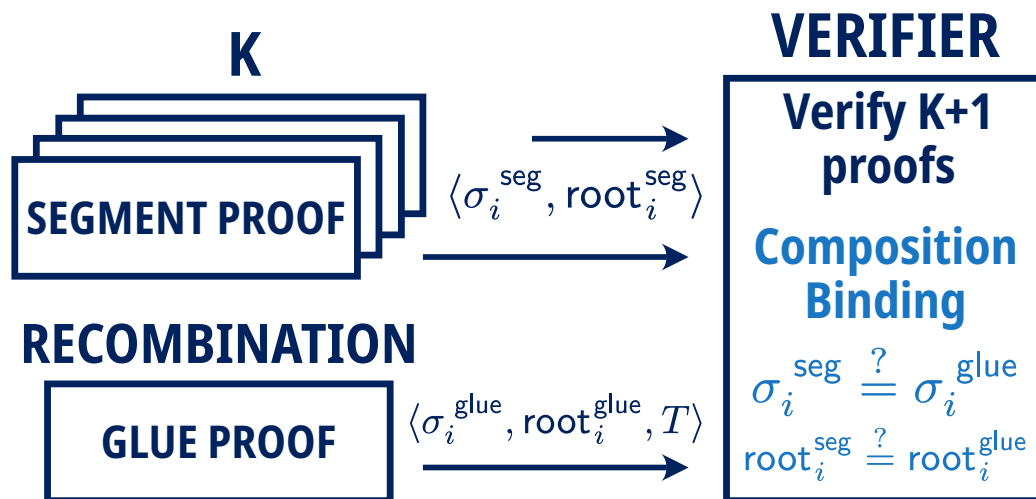
- Sort: M values, exact
- Product fingerprint: Constant size, prob.

Where should recombination be enforced?

External recombination leaves binding to the verifier



External recombination leaves binding to the verifier



Composition Binding

- After $K + 1$ proof verifications
- Requires public summaries

**Plaintext summary values
disclose private information!**

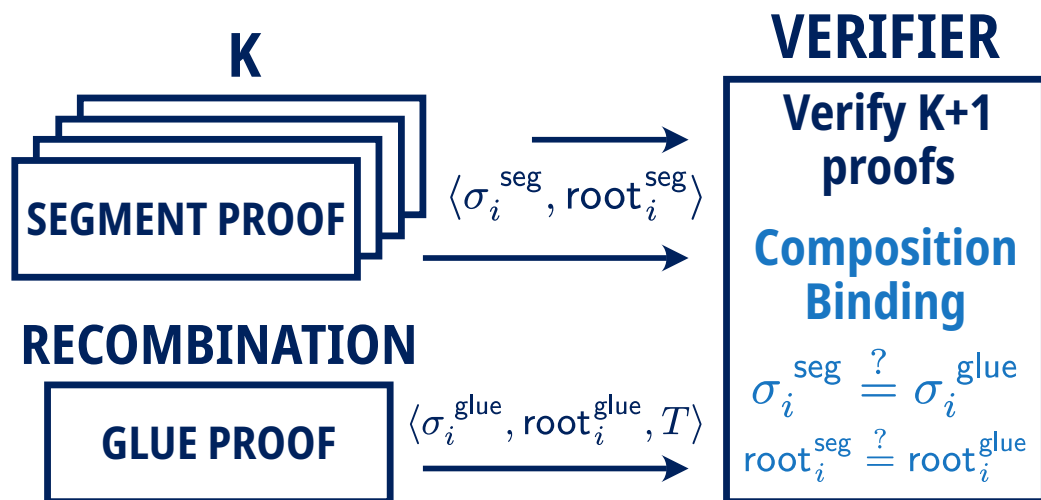
Plaintext summary leakage

$$\sigma_i = (\text{cov}_i, \text{start}_i, \text{end}_i, L_i)$$

- cov_i : members and segment size (sort)
- $\text{start}_i, \text{end}_i$: endpoints $\rightarrow$ boundary arcs
- L_i : partial cost

Across segments: Exact route cost + partition

External recombination leaves binding to the verifier



Composition Binding

- After $K + 1$ proof verifications
- Requires public summaries

Plaintext summary values disclose private information!

Plaintext summary leakage

$$\sigma_i = (\text{cov}_i, \text{start}_i, \text{end}_i, L_i)$$

- cov_i : members and segment size (sort)
- $\text{start}_i, \text{end}_i$: endpoints $\rightarrow$ boundary arcs
- L_i : partial cost

Across segments: Exact route cost + partition

Commit summary + blinding factor

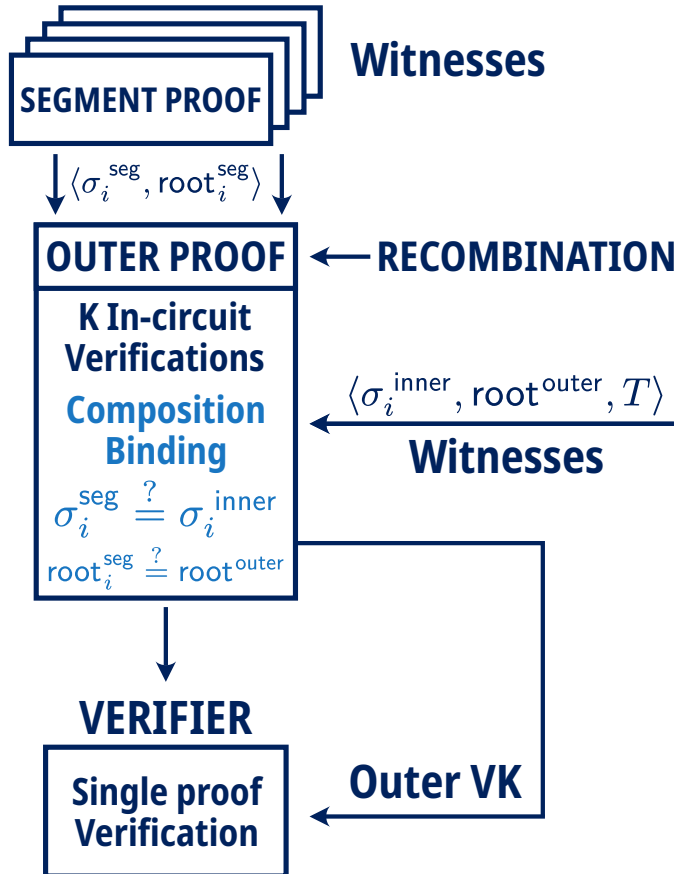
$$D_i = \text{Commit}(\sigma_i; r_i)$$

- The glue proves a private opening (σ_i, r_i)
- The verifier compares digests $D_i^{\text{seg}} = D_i^{\text{glue}}$

We still have $K+1$ proofs $\rightarrow$ Recursion

Recursion restores a single final proof

K INNER PROOFS



Inner segment proofs

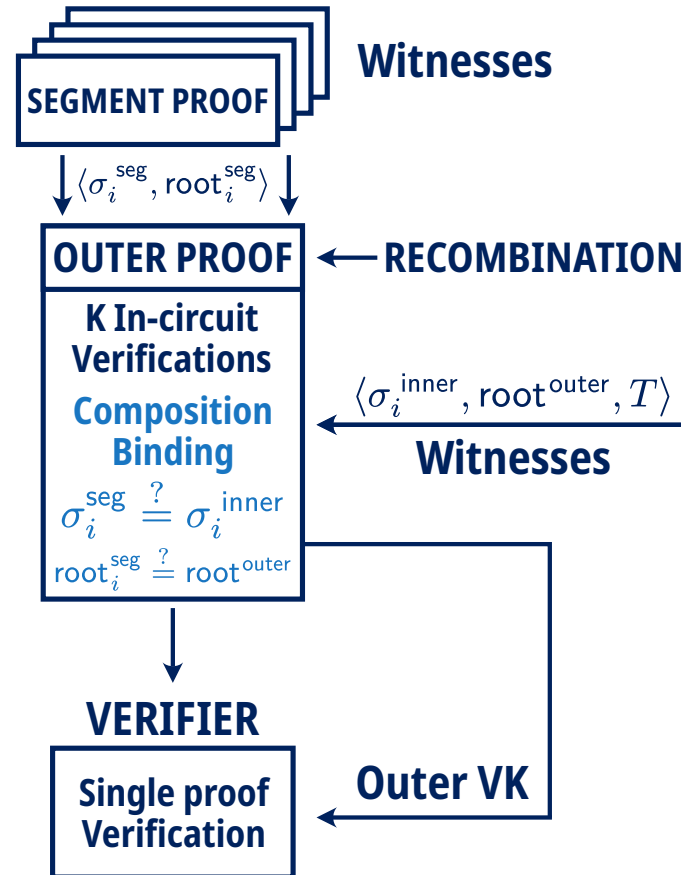
- Never reach the verifier
- Plaintext summaries
- Product fingerprint

Outer proof

- Consumes inner proofs
- Binds inner proofs
- Restores global relation

Recursion restores a single final proof

K INNER PROOFS



Inner segment proofs

- Never reach the verifier
- Plaintext summaries
- Product fingerprint

Outer proof

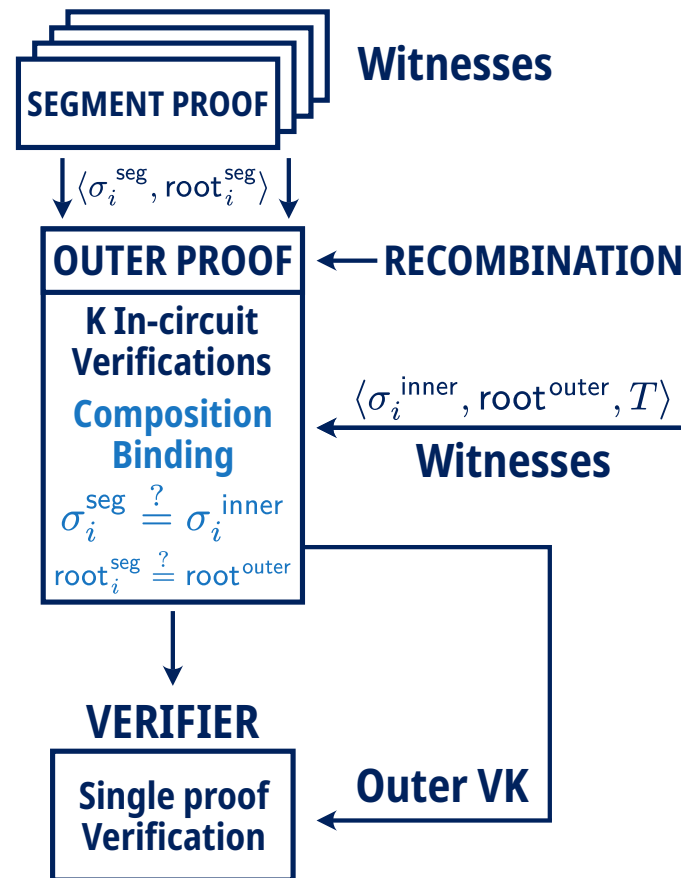
- Consumes inner proofs
- Binds inner proofs
- Restores global relation

In-circuit verification is expensive

- Prover pays the cost of the outer proof
- But the verifier now has a single proof object again

Recursion restores a single final proof

K INNER PROOFS



Inner segment proofs

- Never reach the verifier
- Plaintext summaries
- Product fingerprint

Outer proof

- Consumes inner proofs
- Binds inner proofs
- Restores global relation

In-circuit verification is expensive

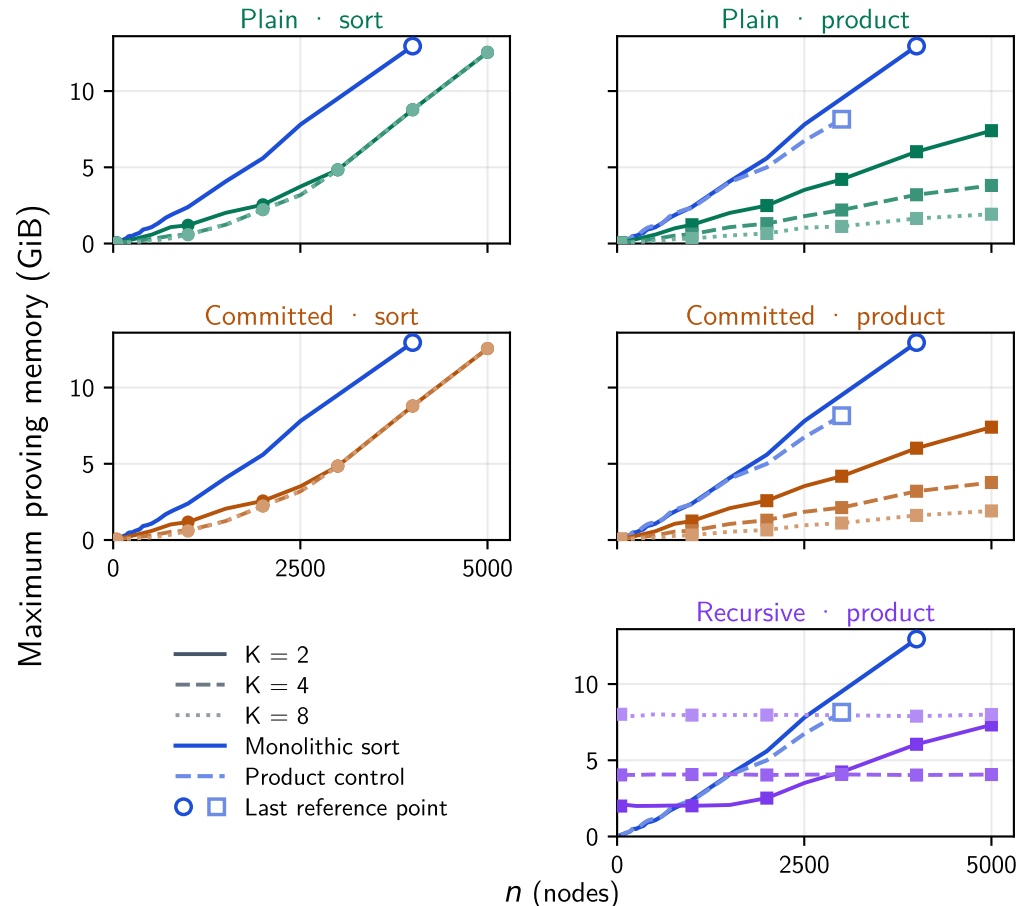
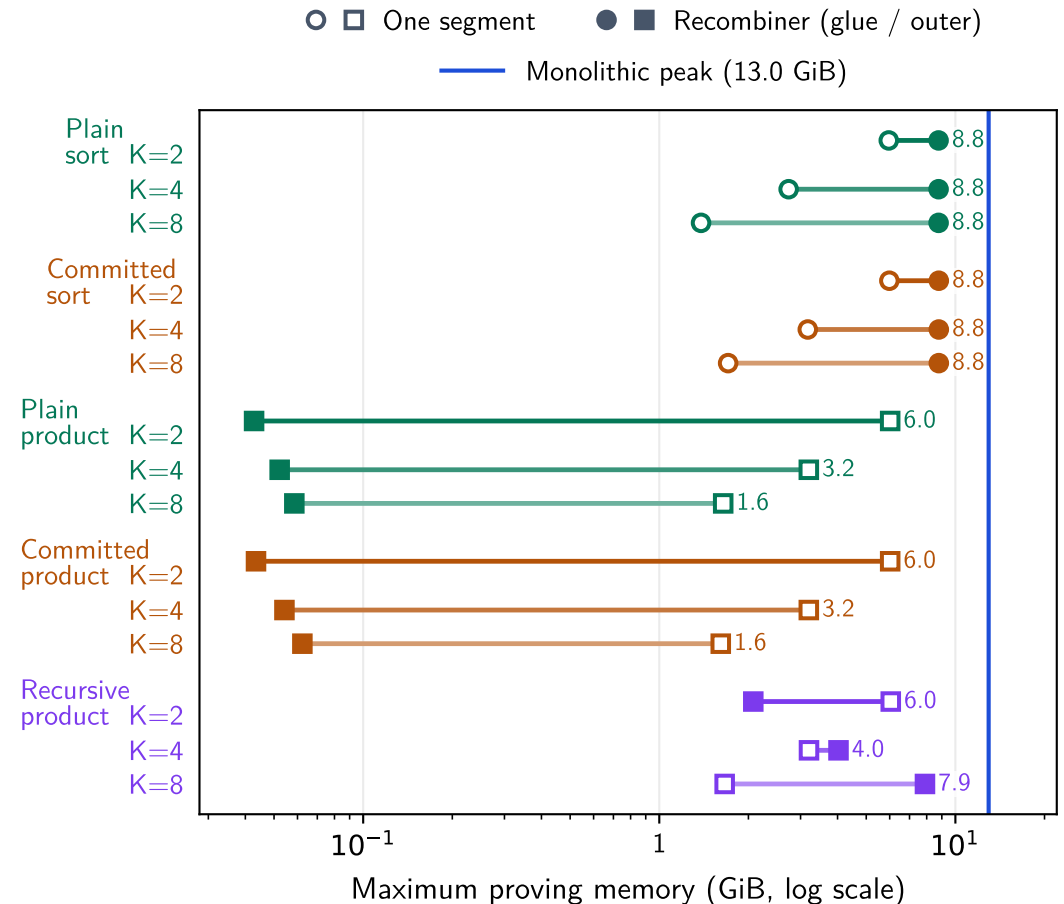
- Prover pays the cost of the outer proof
- But the verifier now has a single proof object again

Recombination strategies have trade-offs of their own

- Proof topology
- Public surface
- Verifier load

But how do composite proofs compare to monolithic ones?

The recombiner determines the memory regime

Peak memory across n · linear axesComponents at $n = 4000$ · logarithmic axis

Sequential proving time accumulates across every stage

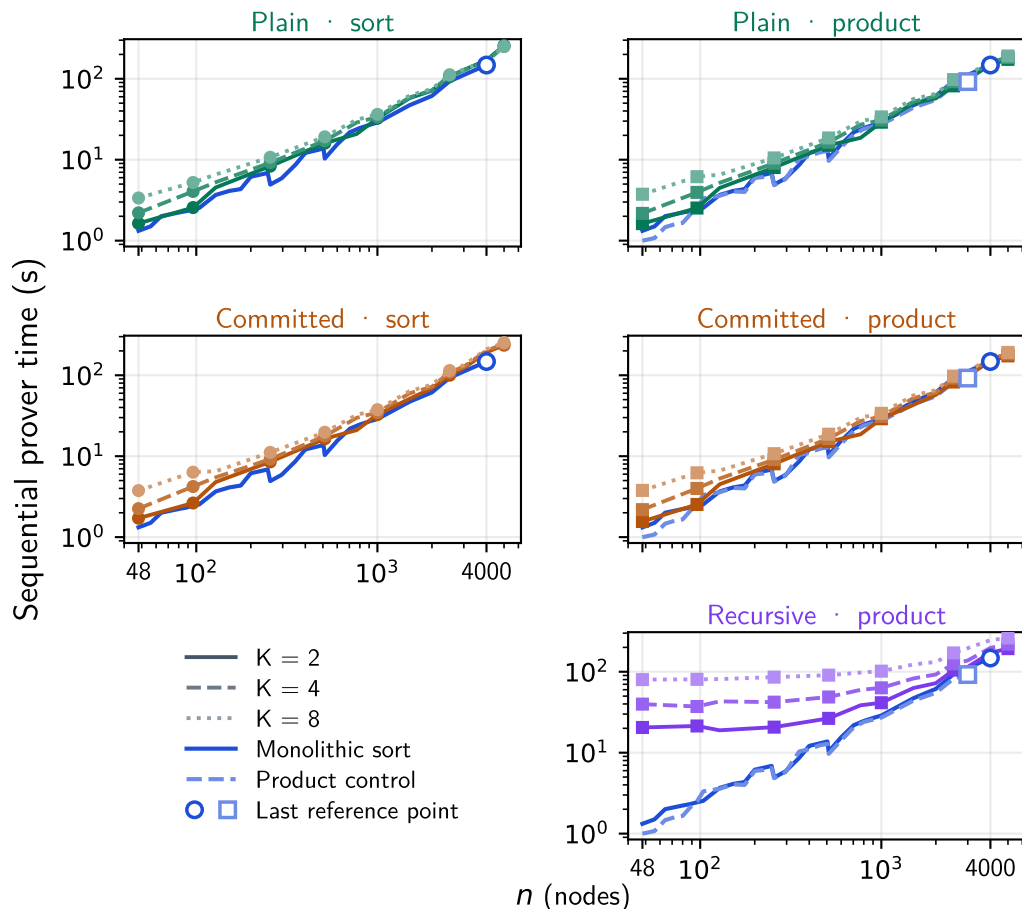


DIPARTIMENTO
MATEMATICA

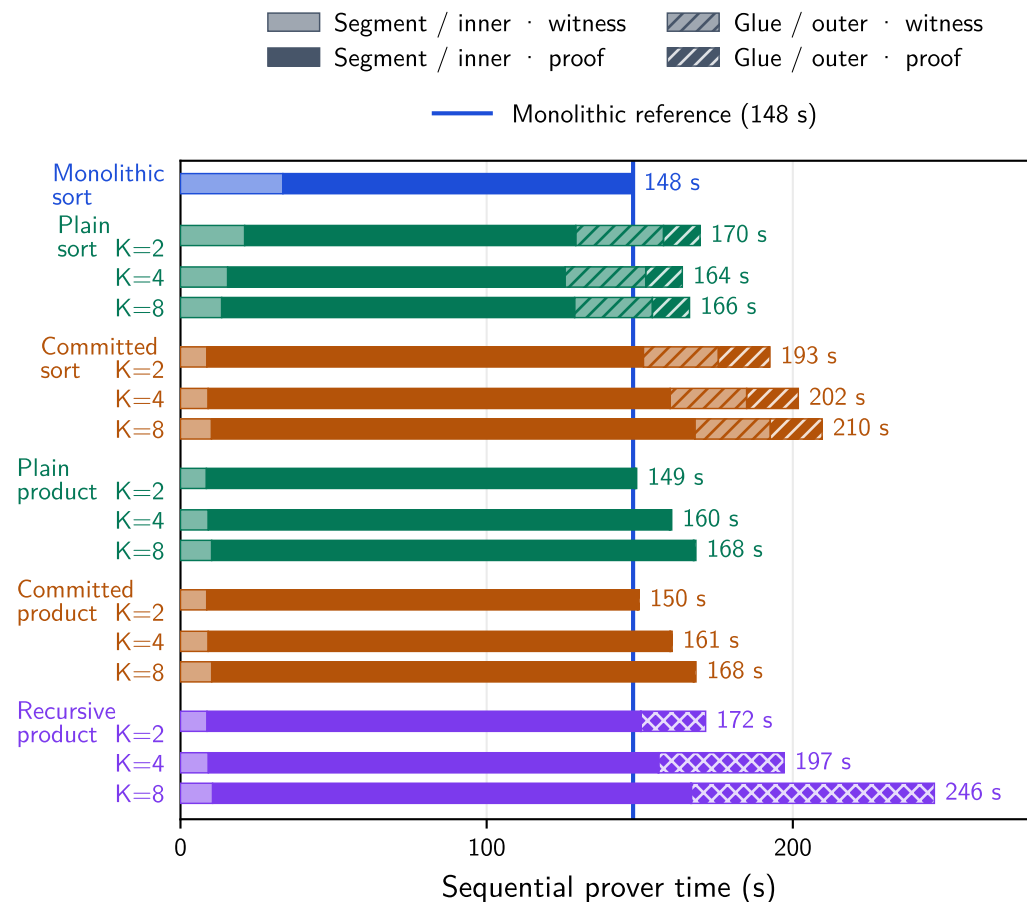


UNIVERSITÀ
DI PADOVA

Sequential prover time across n · logarithmic axes

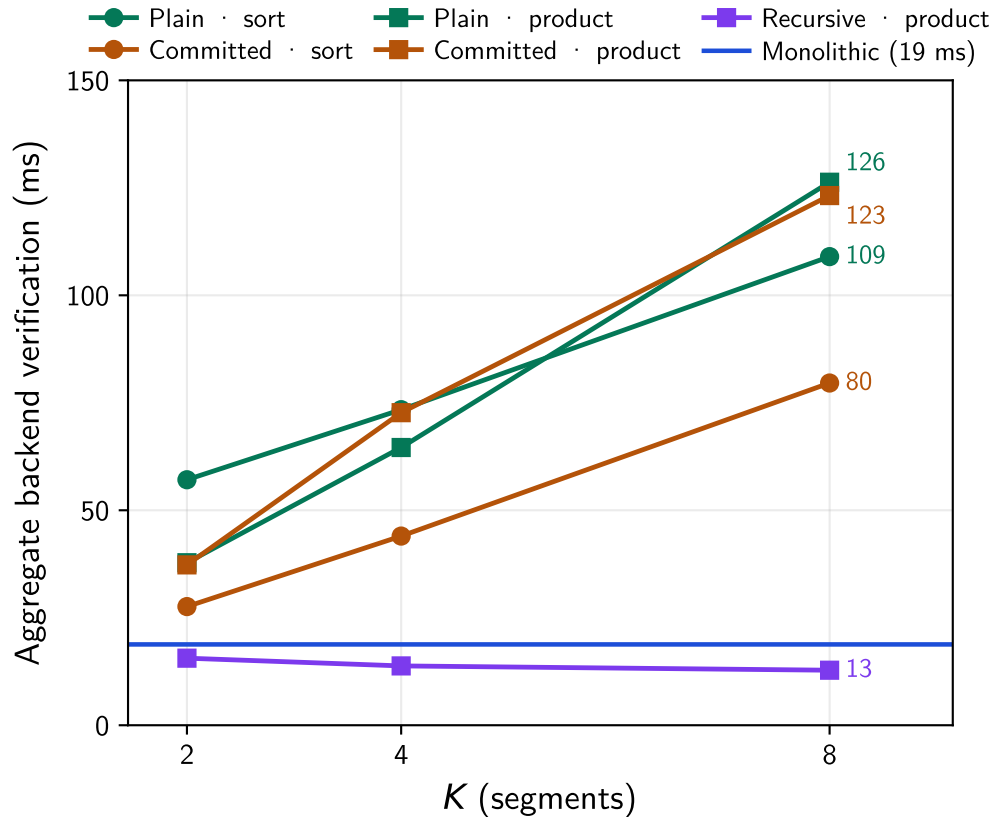


Stages at $n = 4000$ · sequential execution

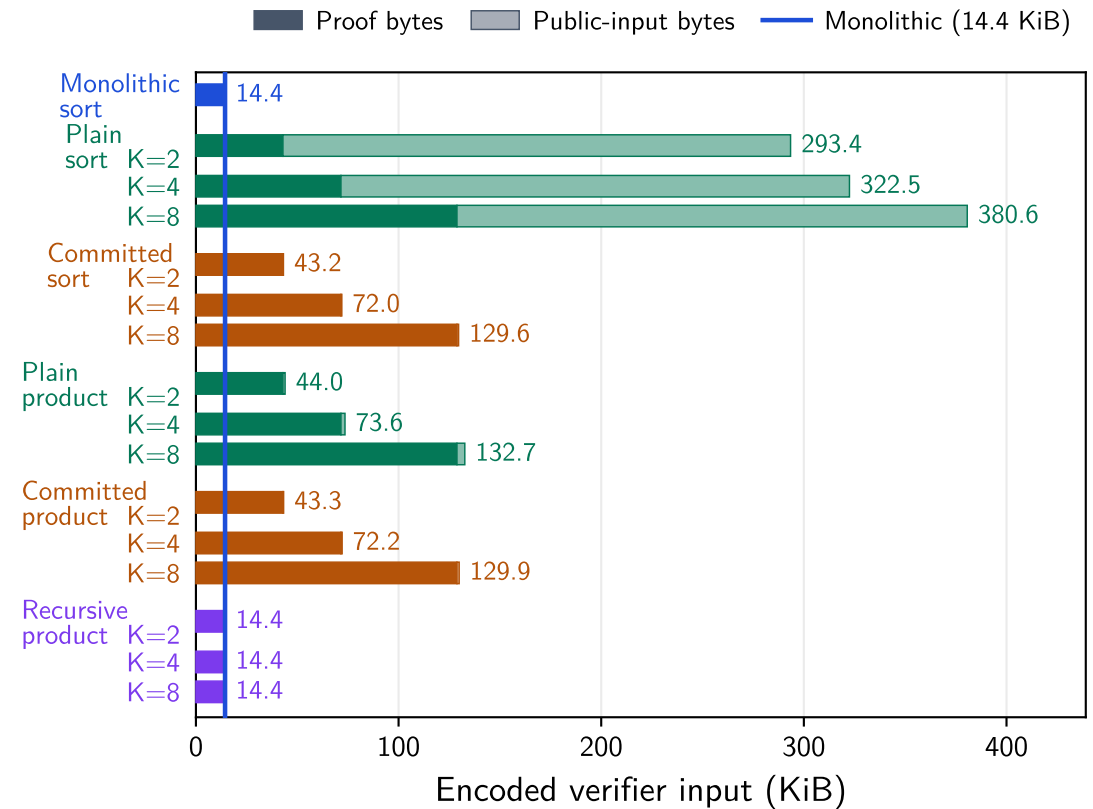


Proof topology moves work to or from the verifier

Final-verifier time at $n = 4000$



Verifier input at $n = 4000$



Architecture determines both prover resources and verifier load

There is no universal winner

Which architecture fits different deployment scenarios?

Whole proof fits in RAM

Monolithic

- One circuit and one final proof.
- Exact coverage with private route and cost.

RAM is limited

Sequential external decomposition

- Lower peak memory through sequential tasks.
- Longer completion, $K + 1$ proofs and reconciliation.

Several workers available

Parallel external decomposition

- Completion follows the slowest segment or glue task.
- $K + 1$ proofs and reconciliation still required.

Small verifier interface

Monolithic if it fits, recursion otherwise

- One final proof.
- Recursion adds a dependent outer proving stage.

There is no universal winner

Which architecture fits different deployment scenarios?

Whole proof fits in RAM

Monolithic

- One circuit and one final proof.
- Exact coverage with private route and cost.

RAM is limited

Sequential external decomposition

- Lower peak memory through sequential tasks.
- Longer completion, $K + 1$ proofs and reconciliation.

Several workers available

Parallel external decomposition

- Completion follows the slowest segment or glue task.
- $K + 1$ proofs and reconciliation still required.

Small verifier interface

Monolithic if it fits, recursion otherwise

- One final proof.
- Recursion adds a dependent outer proving stage.

Increasing K does not guarantee lower memory or faster completion.

Decomposition extends the feasible instance range

Refined baseline

- Sorting and a Merkle root remove avoidable costs.
- One proof, with a growing whole-route proving task.

Decomposition

- Smaller tasks lower peak memory, with added recombination work.
- At $n = 5000$: composites succeed, monolithic runs out of memory.

Completion time

- No sequential speedup at $n = 4000$.
- Parallel gains require extra workers.

Backup

Related work and thesis contribution

Research direction

Representative work Focus

Backend distribution

DIZK, Pianist
Wu et al., 2018
Liu et al., 2024

Distribute the proving work for one circuit

Circuit decomposition

HEKATON
Rosenberg et al., 2024

Prove subcircuits and enforce consistency across their boundaries

Graph-specific proving

LaPathZK
Zhu et al., 2026

Tailor a distributed protocol to a hidden path and cost bound

Semantic Route-level Decomposition

This thesis

Monolithic baseline and composite constructions

Limitations and future work

Limitations

One relation, stack, host, and instance per size

Equal segments, selected K , and estimated parallel execution

Global sorting and recursive verification can become bottlenecks

Prototype assurance and one logical prover

Future work

Test other relations, stacks, hardware, and instance families

Adapt partitions to resources and measure a networked deployment

Investigate cheaper exact coverage and proof composition

Formalize composition security and define worker trust and confidentiality

What carries over beyond TSP?

Design question

In this thesis

Where can the witness be partitioned?

Consecutive route segments

What can each part establish?

Internal arc costs and local coverage evidence

What remains global?

Complete coverage, boundary continuity, and the total bound

What must cross the proof boundary?

Certified coverage evidence, endpoints, and partial costs

Concrete applicability requires a useful partition and manageable recombination

Why TSP, and what is being proved?

Property of the problem

Value for this study

A candidate route can be checked efficiently

Gives a concrete relation to encode in a proof

Internal arcs and partial costs admit local checks

Provides meaningful segment-level proving tasks

Coverage, boundary arcs, and the total bound remain global

Makes the work required by recombination explicit

Claim

Knowledge of a Hamiltonian cycle whose cost is at most the public threshold T

Starting point

The prover already holds a candidate route

Outside the claim

Finding the route, establishing optimality, or demonstrating that a vehicle actually followed it

Cybersecurity motivation and trust boundaries

Security concern

A provider claims compliance with the agreed requirements

The route and exact cost contain sensitive information

Separately valid proofs may describe inconsistent pieces

What the construction provides

Verify coverage and the cost bound against the intended matrix

Protect private witness values, subject to what the chosen interface exposes

Bind the certified segments into one complete route claim

Trust boundary

- The verifier authenticates the matrix root, threshold, and verification key.
- With multiple workers, we assume they operate under one logical prover.

What each architecture reveals

Construction

Proof package and exposed values

Monolithic sort

One proof; no per-segment interface

Plain external

$K + 1$ proofs; endpoints, partial costs, and coverage evidence

Committed external

$K + 1$ proofs and blinded summary commitments; product also exposes X

Recursive product

One outer proof; inner proofs and summaries remain private

- **Plain summaries reveal the exact total cost using the public matrix. Sort also reveals segment membership.**
- **Committed product: public challenge X allows testing a guessed complete ordered route.**

How commitments link hidden summaries

$D_i = \text{Commit}_{r_i}(V_i)$, where V_i : private summary, r_i : secret blinding

Who checks?

Segment circuit

Glue circuit

Final verifier

What is established?

Derives the summary and binds it to its digest

Opens the digest privately and checks recombination

Verifies the proofs and matches the digest copies $D_i^{\text{seg}} = D_i^{\text{glue}}$

Commitment properties

- secret blinding factors $\rightarrow$ Hiding
- matching digests $\rightarrow$ Binding

How the implementation was checked

Valid reference cases accepted, targeted manipulations rejected

Targeted manipulation

Repeat a node and omit another

Alter a cost or authentication path

Combine valid proofs with inconsistent summaries

Alter an inner proof or substitute the wrong relation

Rejection boundary

Coverage constraints

Merkle index and root checks

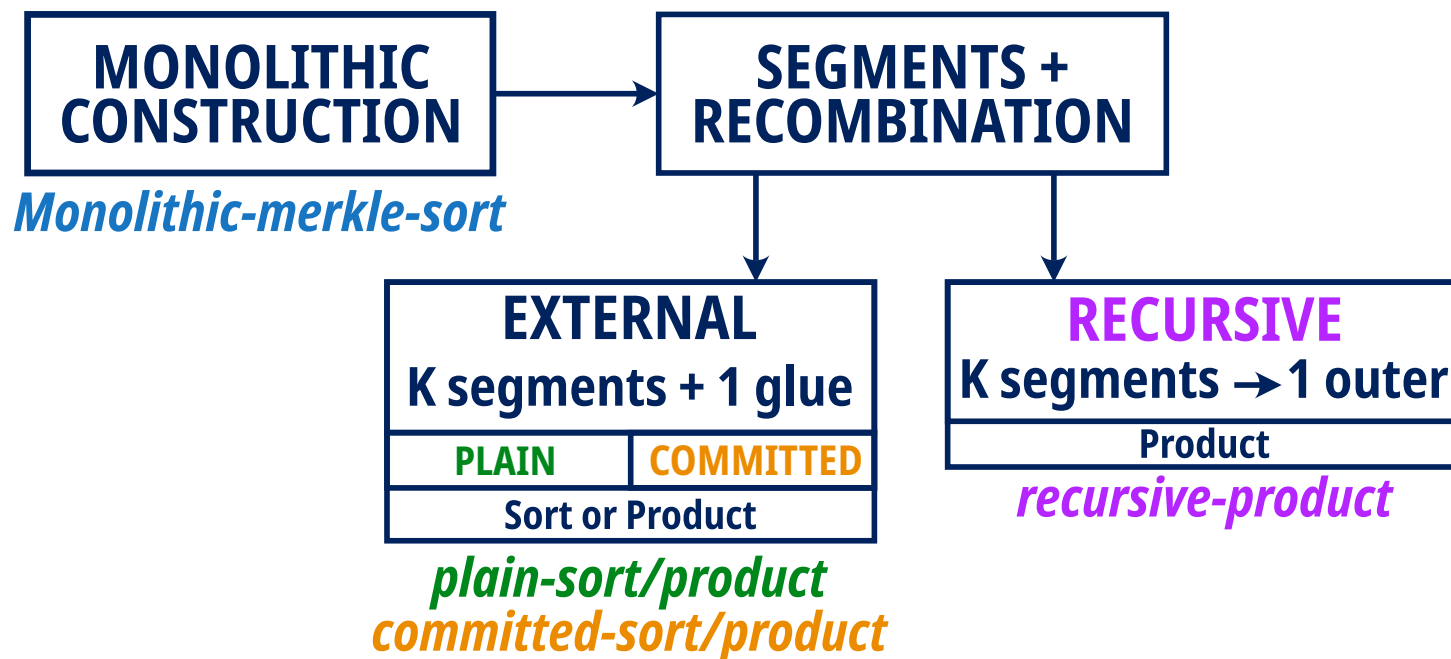
External reconciliation

Recursive verification and key binding

A controlled evaluation isolates architectural effects



Decomposition



Fixed stack & Hardware

- Noir + Barretenberg
- Consumer laptop
 - i7 4C, 8T
 - 16 GiB RAM

Instance sizes

- Route: Up to 5000 nodes
- Segments: $K \in \{2, 4, 8\}$

What the performance comparison measures

Task time = witness generation + proof generation

Schedule

Completion time

Sequential external

Sum of segment tasks plus glue

Fully parallel external

Longest segment or glue task

Sequential recursive

Sum of inner tasks plus outer

Fully parallel recursive

Longest inner task, then outer

Measurement boundary

Parallel: projected from isolated runs using additional comparable workers.

Memory: largest Barretenberg-reported task peak, not total concurrent memory.

Online time: excludes preprocessing, setup, final verification, and network coordination.

What each comparison tells us

Comparison

Monolithic product / decomposed product

Plain / committed, within one coverage family

External / recursive product

Sort / product

What changes

Partitioning and recombination under the same coverage mechanism

Hiding summaries and checking their openings

Moving proof checks and binding into the outer circuit; changing final disclosure

Coverage representation and guarantee

Why valid segment proofs are insufficient

Example: six target nodes $\{0, 1, 2, 3, 4, 5\}$, two segments.

Failure	Example	Required check
Global coverage	$R_0 = (0, 1, \mathbf{2}), \quad R_1 = (\mathbf{2}, 4, 5)$ Node 2 repeats; node 3 is missing	Check coverage across all segments
Composition binding	Segment certifies endpoint 2 Glue uses endpoint 3	Match the glue's summary to the certified segment summary

External composition: the verifier reconciles summaries or commitments.

Recursion: the outer circuit verifies inner proofs and reconciles their certified inputs.

Acceptance requires valid local proofs, global route checks, and agreement between them.

What sort and product pass to recombination

Shared inputs

Boundary endpoints, internal cost, and matrix root: $(\text{start}_i, \text{end}_i, L_i, \text{root})$

Sort

$M + 4$ public inputs (Exact)

COVERAGE
EVIDENCE

$$S_i = \text{sort}(R_i)$$

Product

9 public inputs (Probabilistic)

$$P_i = \prod_{j=0}^{M-1} (X + R_{i[j]})$$

PUBLIC
INTERFACE

M sorted node identifiers.

$$(P_i, h_i^{\text{in}}, h_i^{\text{out}}, h^{\text{term}}, X)$$

RECOMBINATION

Glue sorts the combined lists.

Anchors join the route hash chain.

$$X = H(h^{\text{term}})$$

Why sorting was selected for the baseline

Mechanisms considered

Pairwise distinctness	Quadratic circuit and witness work
Indicator vector	Fast witness generation; read-write memory
Inverse map	Strongest measured monolithic candidate
Sorting	Same exact mechanism across proof stages

Sort provides same exact check everywhere

Monolithic

$$\text{sort}(\pi) = (0, \dots, n - 1)$$

Segment

$$\text{sort}(R_i) = S_i$$

Recombination

$$\begin{aligned} \text{sort}(S_0 \parallel \dots \parallel S_{K-1}) &= \\ &= (0, \dots, n - 1) \end{aligned}$$

How the Merkle root authenticates costs

Example: $n = 6$, selected arc $2 \rightarrow 5$, claimed cost $C[2, 5] = 7$.

Computed root z_6

$$\uparrow z_6 = H(z_5, s_5)$$

$$\uparrow z_5 = H(s_4, z_4)$$

$$\uparrow z_4 = H(z_3, s_3)$$

$$\uparrow z_3 = H(z_2, s_2)$$

$$\uparrow z_2 = H(z_1, s_1)$$

$$\uparrow z_1 = H(s_0, z_0)$$

Leaf 17: $z_0 = 7$

$s_0, \dots, s_5$: sibling hashes

1. Selected position

Route endpoints require leaf $2 \cdot 6 + 5 = 17$.

2. Opened position

Path bits reconstruct the same index: $1 + 16 = 17$.

3. Authenticated value

Recompute the root from cost 7 and its siblings; require equality with the agreed public root.

Both checks matter: the cost must belong to the selected arc and the agreed matrix.

The root compresses the proof's public interface. The matrix remains public problem data.

Product coverage and the Schwartz–Zippel bound

Schwartz–Zippel

$$D \in \mathbb{F}[X_1, \dots, X_m], \quad D \neq 0$$

$$\deg D = d, \quad \mathbf{r} \in S^m \text{ uniformly}, \quad S \subseteq \mathbb{F}$$

$$\Pr[D(\mathbf{r}) = 0] \leq \frac{d}{|S|}$$

Polynomial identity test

$$P \neq Q \implies D = P - Q \neq 0$$

$$P(\mathbf{r}) = Q(\mathbf{r}) \iff D(\mathbf{r}) = 0$$

Product coverage in this thesis

$$F_\pi(Z) = \prod_{j=0}^{n-1} (Z + \pi_j)$$

$$F_{\text{target}}(Z) = \prod_{v=0}^{n-1} (Z + v)$$

$$\deg(F_\pi - F_{\text{target}}) \leq n - 1$$

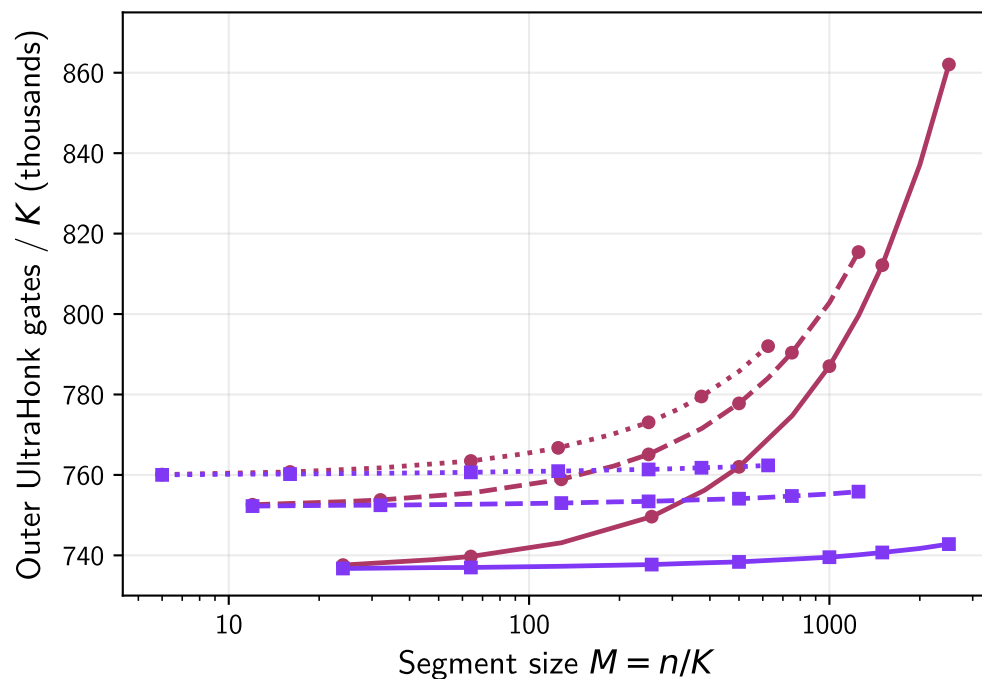
$$F_\pi \neq F_{\text{target}} \implies \Pr[F_{\pi(X)} = F_{\text{target}(X)}] \leq \frac{n-1}{q}$$

$$X = H(h^{\text{term}}) \in \mathbb{F}_q$$

Why product was selected for recursion

Recursive-sort Recursive-product $K = 2$ $K = 4$ $K = 8$

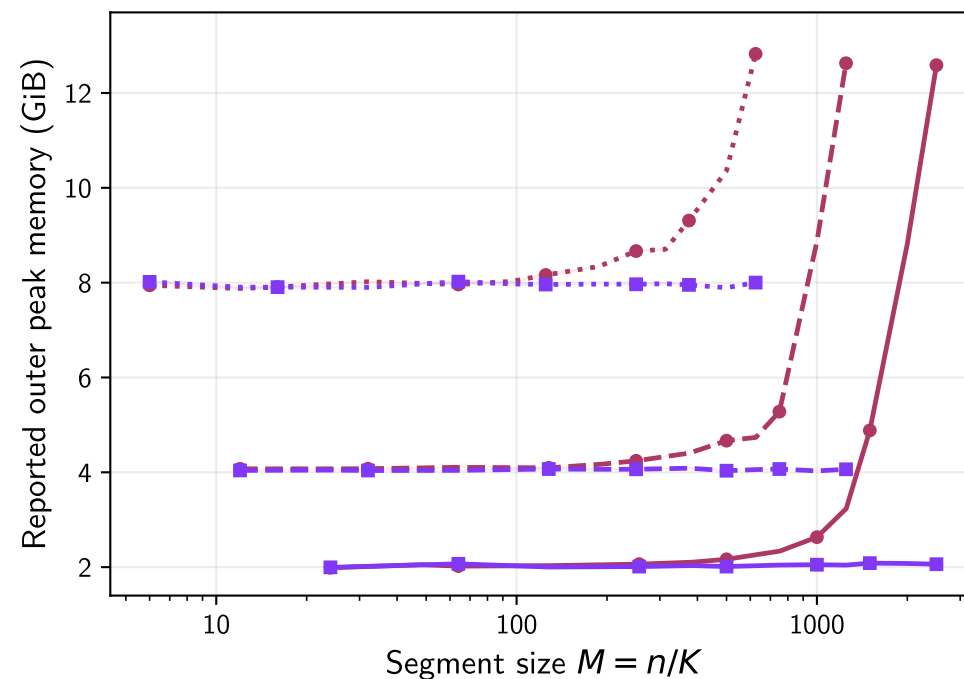
Outer circuit work per segment



Inner inputs per segment: sort $M + 4$; product 9

Product retains probabilistic coverage

Outer prover memory

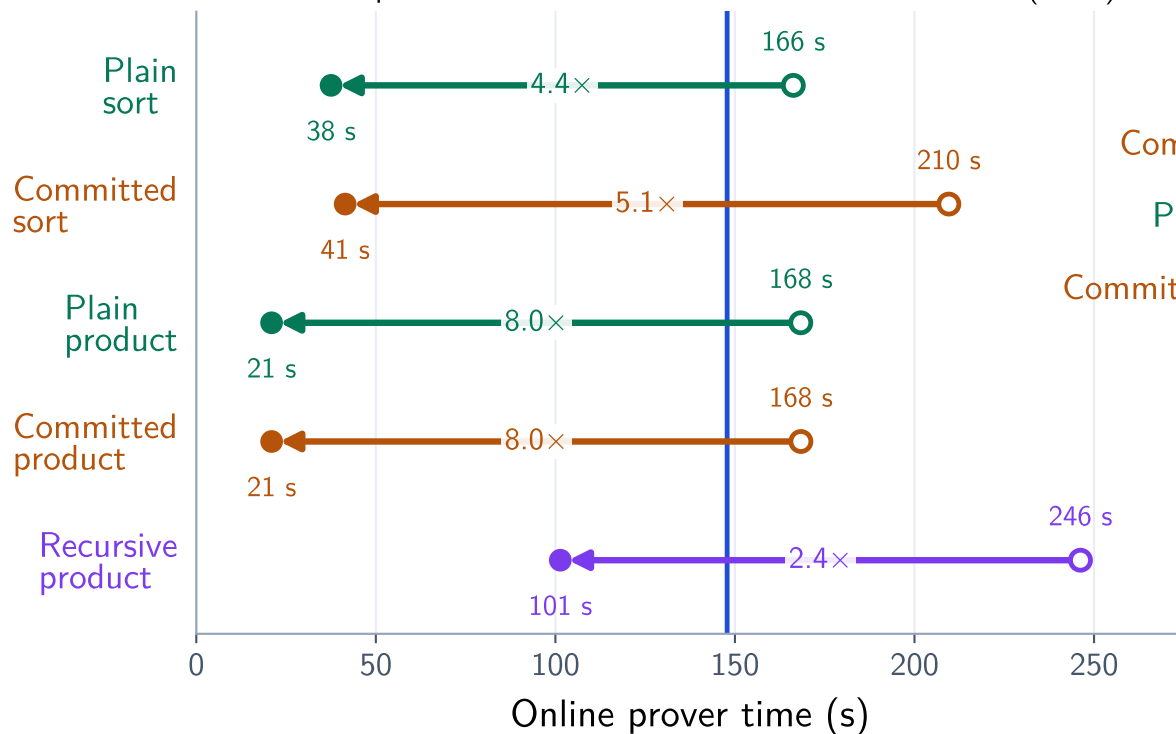


Extra workers unlock parallel speedup

Sequential versus ideal distributed time

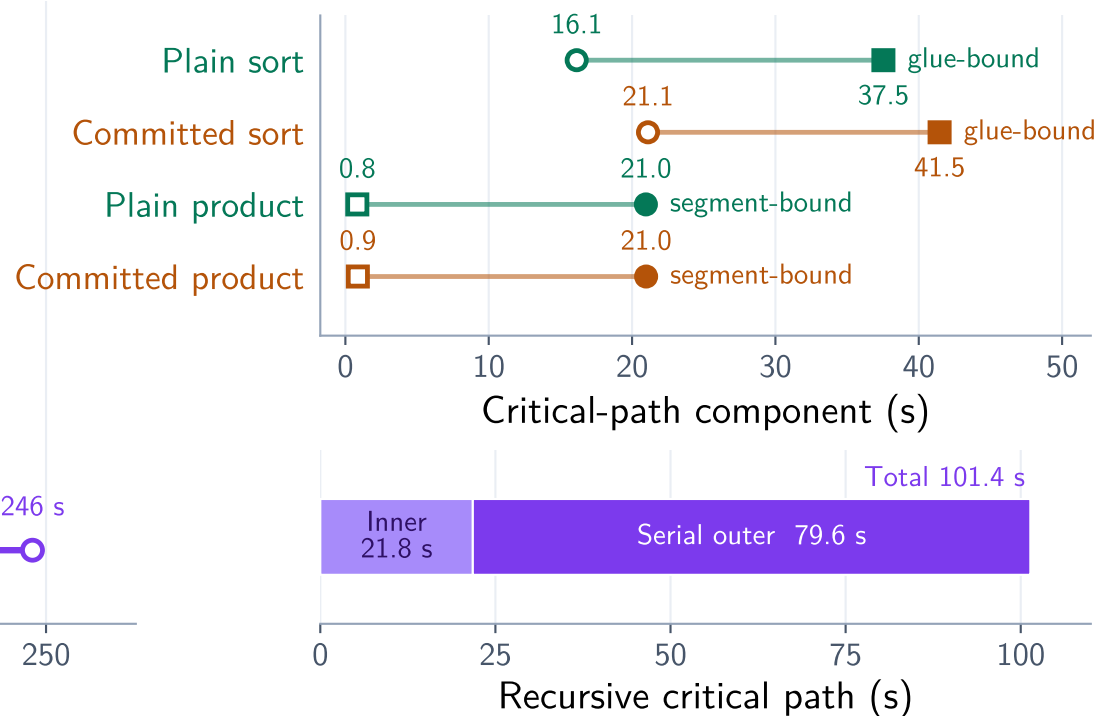
 $n = 4000, K = 8$

○ Sequential ● Ideal distributed — Monolithic (148 s)



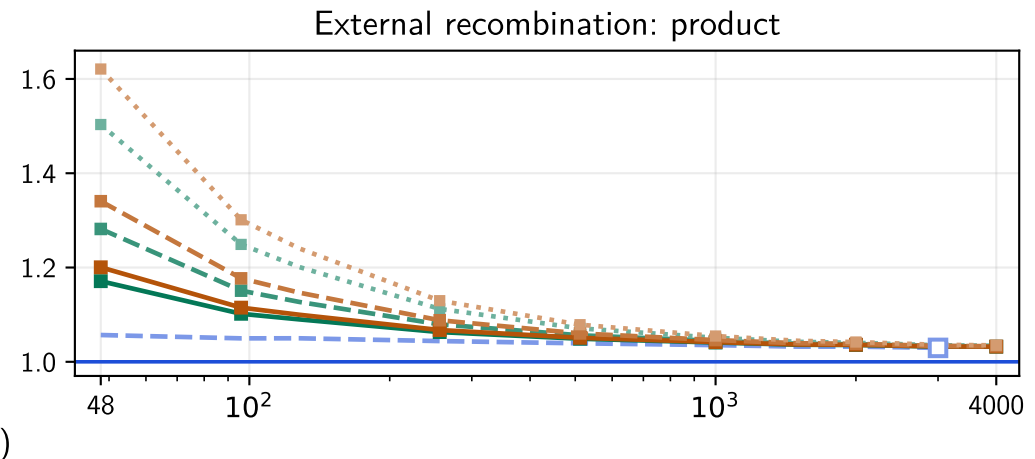
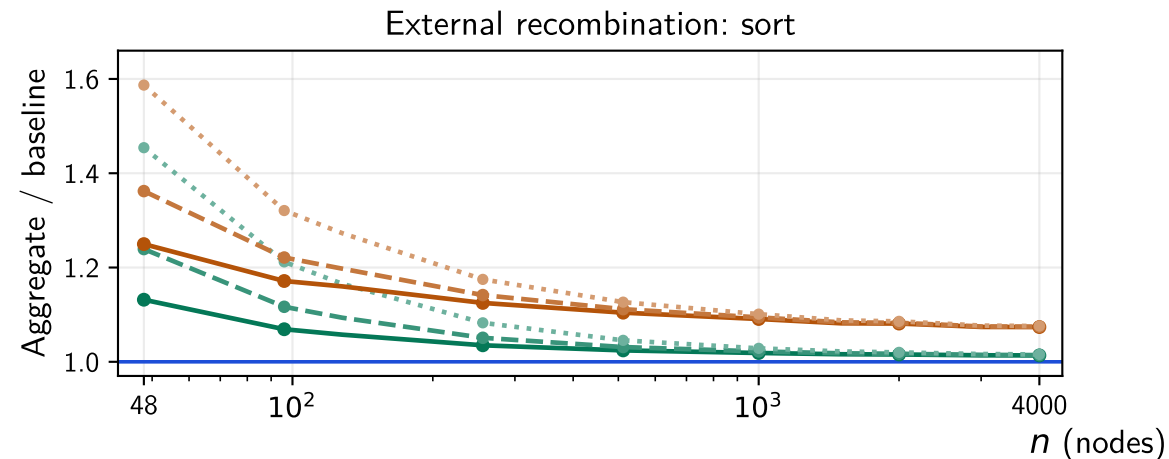
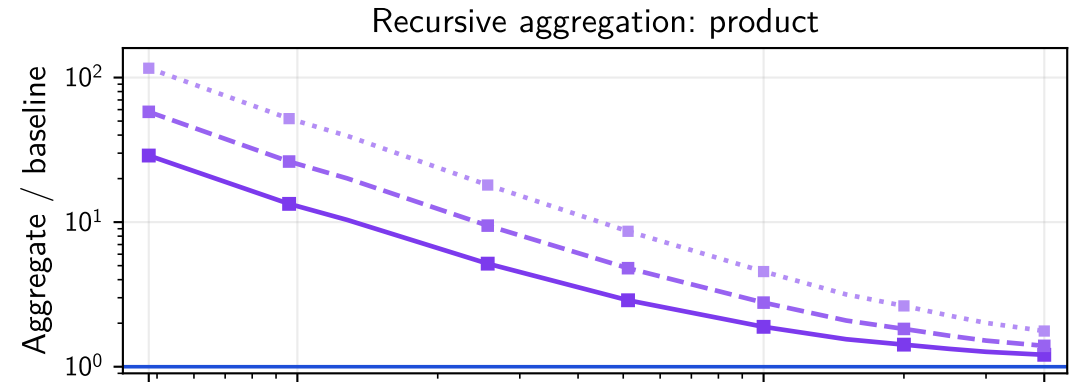
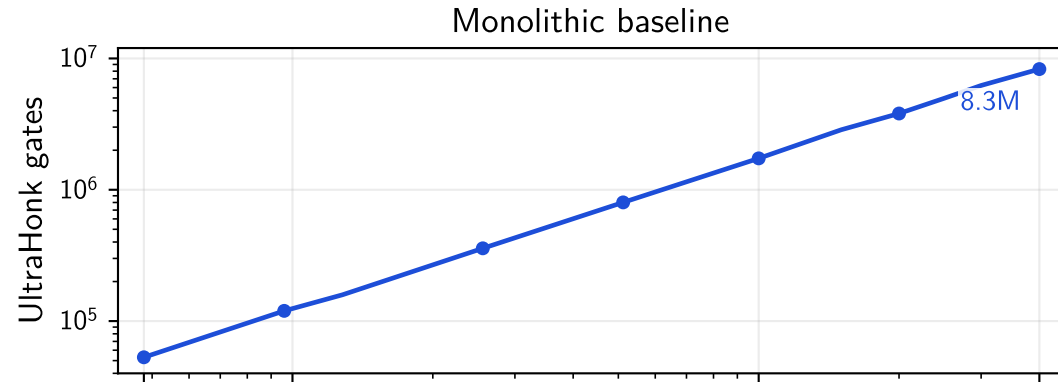
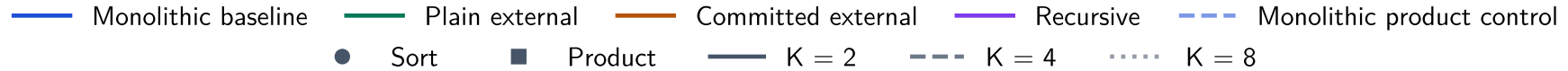
Why the speedups differ

○ Slowest segment □ Recombination



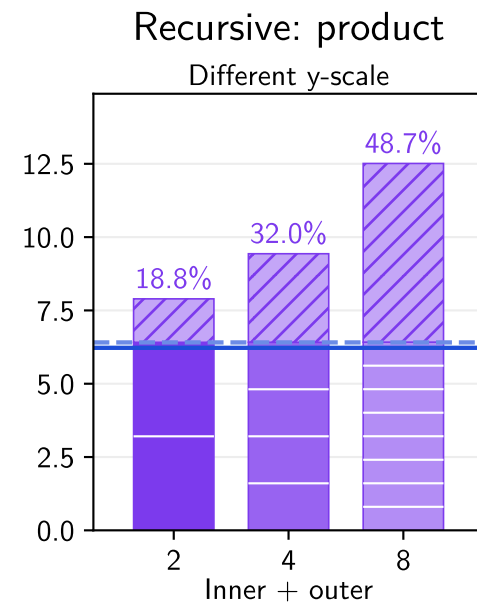
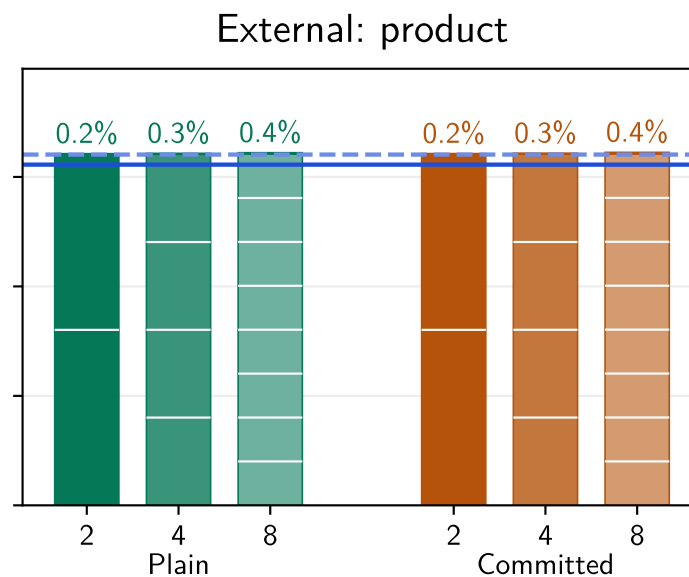
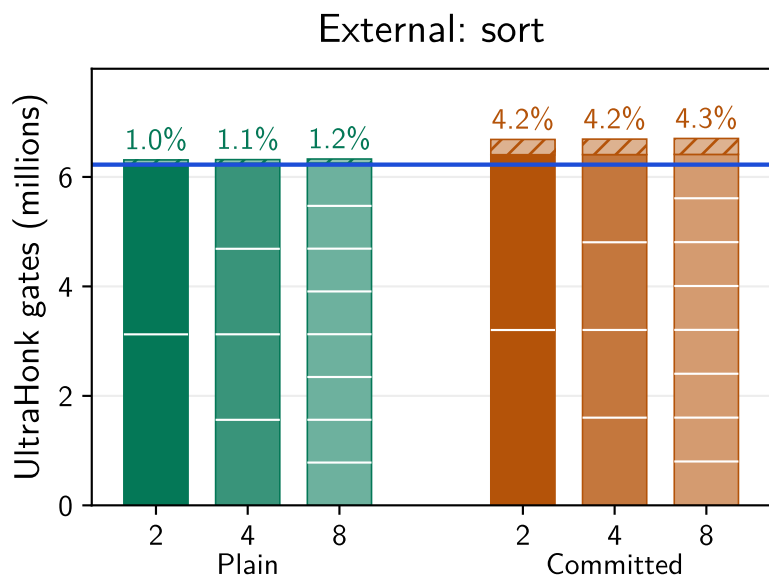
Decomposition makes proving feasible. Extra workers can then make it faster.

Decomposition redistributes work rather than removing it



How circuit work is distributed across proving tasks

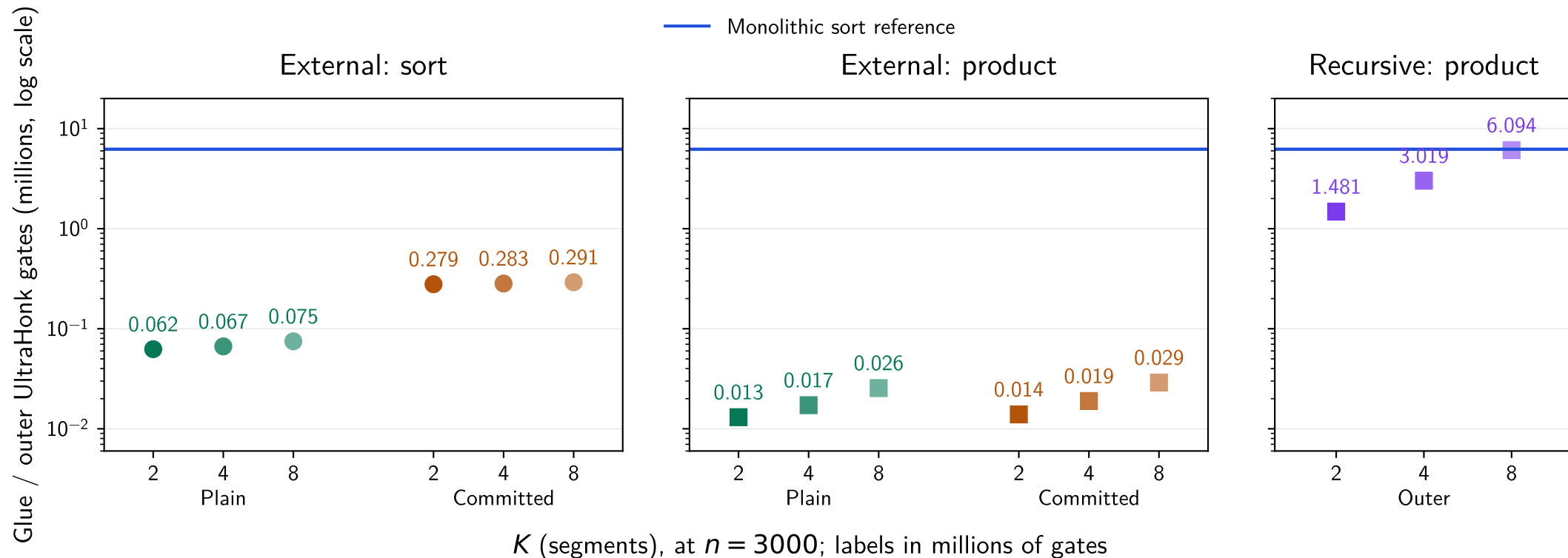
Segments: one slab per task Monolithic sort
Recombiner; labels show share Monolithic product



K (segments), at $n = 3000$

Smaller tasks retain most work; recombination adds overhead.

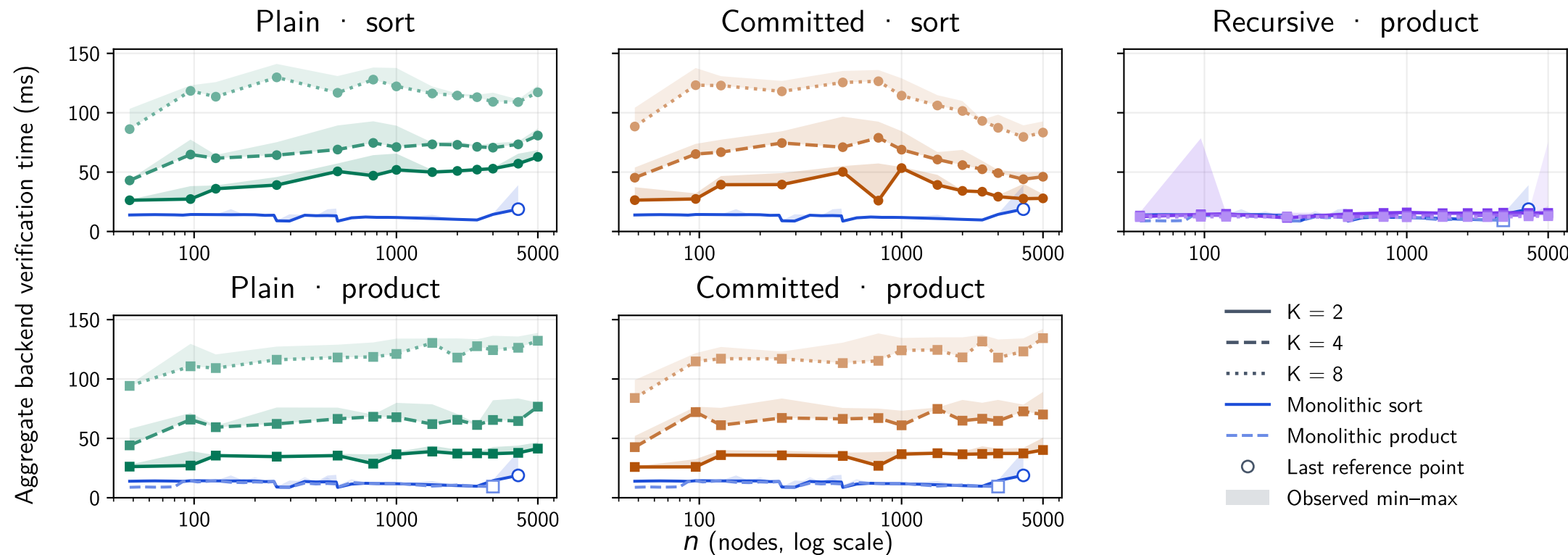
The cost of recombination



Summary width and in-circuit verification shape recombination cost.

External proof verification and reconciliation are excluded.

Verification time across instance sizes



Backend commands include startup/loading; application reconciliation excluded.